Министерство цифрового развития, связи и массовых коммуникаций
Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования «Сибирский государственный университет
телекоммуникаций и информатики»
(СибГУТИ)

Кафедра прикладной математики и кибернетики

Операционные системы

Лабораторная работа №3

Выполнил: Студенты 3-го курса,
группы ИП-111
Гердележов Даниил Дмитриевич
Махаева Алина Витальевна
Клепче Георгий Васильевич

Проверил преподаватель:
Малков Евгений Александрович

Новосибирск, 2023

Цель: знакомство с процессами Linux.

Упражнение 1. Протестируйте программы, рассмотренные на Лекции 3.

Упражнение 2. Создайте процесс с помощью вызова fork, с помощью команд ps и grep получите информацию о созданных вами родительском и дочернем процессах. Используя команду kill убейте родительский процесс, продолжил ли выполняться дочерний процесс?

Упражнение 3. Создайте дерево процессов с помощью вызова fork. С помощью команды pstree найдите поддерево созданных процессов. В каталоге /proc виртуальной файловой системы найдите папки с именами, совпадающими с идентификаторами созданных процессов, и просмотрите содержимое папок task/children.

Ход работы:

Упражнение 1:

Программа 1:

```c
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>

void oldman();
void recreation();

void oldman()
{
    fprintf(stdout, "I'm not yet dead! My ID is %i\n", (int)getpid());
}

void recreation()
{
    fprintf(stdout, "Who I am? My ID is %i\n", (int)getpid());
}

int main()
{
    pid_t child_pid, parent_pid;
    int i = 0;
    fprintf(stdout, "Before RECREATION %i\n",
            parent_pid = (int)getpid());
    child_pid = fork();
    while (i++ < 5)
        if (child_pid != 0)
            oldman();
        else
            recreation();
    return 0;
}
```

Результат работы программы №1:

```
nothomepc@DESKTOP-JB25TSP:~$ ./app
Before RECREATION 6478
I'm not yet dead! My ID is 6478
I'm not yet dead! My ID is 6478
I'm not yet dead! My ID is 6478
I'm not yet dead! My ID is 6478
I'm not yet dead! My ID is 6478
Who I am? My ID is 6479
Who I am? My ID is 6479
Who I am? My ID is 6479
Who I am? My ID is 6479
Who I am? My ID is 6479
```

Эта программа использует системные вызовы для создания процессов с использованием функции **fork()**. Она создает родительский процесс и дочерний процесс, которые выполняют разные функции. Родительский процесс вызывает функцию **oldman()**, а дочерний процесс вызывает функцию **recreation()**. Оба процесса будут выводить свой идентификатор процесса (PID) на стандартный вывод.

Вот что будет происходить в этой программе:

1. Родительский процесс будет вызывать **oldman()** и выводить свой PID на стандартный вывод.
2. Дочерний процесс будет вызывать **recreation()** и также выводить свой PID на стандартный вывод.
3. Оба процесса будут выполняться в цикле, который будет повторяться 5 раз, и они будут поочередно выводить сообщения с PID на стандартный вывод.

Программа 2:

```c
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
int main()
{
    pid_t child_pid, parent_pid;
    double s = 0.0;
    child_pid = fork();
    if (child_pid != 0)
    {
        s += 3.14;
        fprintf(stdout, "CHILD: %i s=%g &s=%u\n", (int)getpid(), s, &s);
    }
    else
    {
        s += 2.72;
        fprintf(stdout, "PARENT: %i s=%g &s=%u\n", (int)getpid(), s, &s);
    }
    return 0;
}
```

Результат работы программы №2:

```
nothomepc@DESKTOP-JB25TSP:~$ ./app
CHILD: 7413 s=3.14 &s=6859456
PARENT: 7414 s=2.72 &s=6859456
```

Этот программа использует функцию **fork()** для создания дочернего процесса. Он объявляет переменную **s**, которая равна 0.0, затем изменяет значение **s** в родительском и дочернем процессах и выводит значение переменной **s**, а также адрес переменной **s** на стандартный вывод.

Вот что будет происходить в этой программе:

1. Родительский процесс (**child_pid** не равен 0) увеличивает значение **s** на **3.14** и выводит сообщение о значении **s** и адресе переменной **s**.
2. Дочерний процесс (**child_pid** равен 0) увеличивает значение **s** на **2.72** и также выводит сообщение о значении **s** и адресе переменной **s**.

Оба процесса будут работать независимо друг от друга, и значения **s** в них будут разные.

Программа 3:

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
int main()
{
    pid_t child_pid;
    pid_t parent_pid;
    double s = 0.0;
    ;
    FILE *fp;
    child_pid = fork();
    fp = fopen("test.dat", "a+");
    if (child_pid != 0)
    {
        s += 3.14;
        fprintf(fp, "CHILD: %i s=%g &s=%u fp=%u\n", (int)getpid(),
                s, &s, fp);
    }
    else
    {
        s += 2.72;
        fprintf(fp, "PARENT: %i s=%g &s=%u fp=%u\n", (int)getpid(),
                s, &s, fp);
    }
    fclose(fp);
    return 0;
}
```

Результат работы программы №3:

Файл test.dat:

test.dat

```
1   PARENT: 4635 s=2.72 &s=2827957256 fp=1019183776
2   CHILD: 4634 s=3.14 &s=2827957256 fp=1019183776
```

Упражнение 2:

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t child_pid;

    // Создаем дочерний процесс
    child_pid = fork();

    if (child_pid == -1) {
        perror("Ошибка при вызове fork");
        exit(EXIT_FAILURE);
    }

    if (child_pid == 0) {
        // Код, выполняемый дочерним процессом
        printf("Это дочерний процесс (PID: %d)\n", getpid());
        sleep(10); // Пусть дочерний процесс подождет некоторое время
        printf("Дочерний процесс завершился\n");
    } else {
        // Код, выполняемый родительским процессом
        printf("Это родительский процесс (PID: %d)\n", getpid());

        // Используем команду ps и grep для поиска информации о процессах
        system("ps aux | grep my_program");

        kill(getpid(), SIGKILL);
        printf("Родительский процесс завершился\n");
    }

    return 0;
}
```

Результат работы программы:

```
nothomepc@DESKTOP-JB25TSP:~$ ./app
Это родительский процесс (PID: 9132)
Это дочерний процесс (PID: 9133)
nothome+    9134  0.0  0.0   2888   968 pts/5    S+   18:45   0:00 sh -c ps aux | grep my_program
nothome+    9136  0.0  0.0   4024  2044 pts/5    S+   18:45   0:00 grep my_program
Killed
nothomepc@DESKTOP-JB25TSP:~$ Дочерний процесс завершился
```

После выполнения кода родительский процесс будет убит, и дочерний процесс также завершит выполнение, так как родительский процесс больше не будет его удерживать.

Упражнение 3:

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t child_pid;
    int i;

    for (i = 0; i < 3; i++)
    {
        child_pid = fork();
        if (child_pid == 0)
        {
            printf("Дочерний процесс %d с PID %d\n", i + 1, getpid());
            break;
        }
        else if (child_pid < 0)
        {
            perror("fork");
            exit(1);
        }
    }

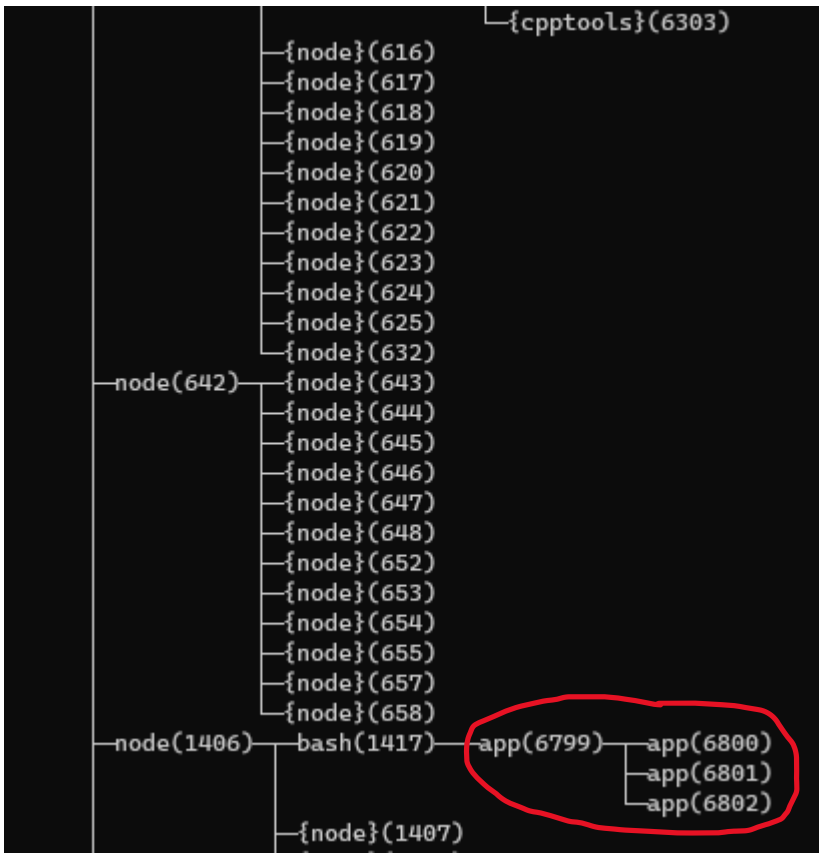
    if (child_pid > 0)
    {
        printf("Родительский процесс с PID %d\n", getpid());
    }

    sleep(99999);
    return 0;
}
```

Результат работы программы:

```
nothomepc@DESKTOP-JB25TSP:~$ ./app
Дочерний процесс 1 с PID 6800
Дочерний процесс 2 с PID 6801
Родительский процесс с PID 6799
Дочерний процесс 3 с PID 6802
```

Создалось 4 процесса: 1 родительский и 3 дочерних.
Теперь найдем процессы в дереве процессов с помощью команды **pstree.**



Созданные процессы в дереве процессов.

Полный путь к созданным процессам:

```
nothomepc@DESKTOP-JB25TSP:/proc/5493$ pstree 6799 -p -s
systemd(1)─┬─init-systemd(Ub(2)─┬─SessionLeader(448)─┬─Relay(450)(449)─┬─sh(450)─┬─sh(451)─┬─sh(482)─┬─node(486)─┬─node(1406)─┬─bash(1417)─┬─app(6799)─┬─app(6800)
                                                
```

Вывод:

- Мы научились работать с процессами в linux;
- Протестировали программы, рассмотренные на Лекции 3;
- создавали процессы и получать информацию с помощью языка программирования си;
- научились смотреть дерево процессов и находить информацию о нужном процессе;
- узнали, что находится в каталоге процессов /proc и в каталоге отдельного процесса.